

Project Overview

Over de komende vijf middagen bouw je je eigen **REST API**. Je begint met de eerste resources en endpoints, en bouwt daar stap voor stap op verder: volledige CRUD, nette foutafhandeling, API design principes, authenticatie en autorisatie. En uiteraard, op de laatste dag presenteer je het eindresultaat aan een groter publiek.

Je kiest zelf een domein voor je API. Denk aan iets wat je zou willen gebruiken. Het kan van alles zijn: een Pokémon encyclopedie, een kaaswinkel, een filmarchief, een receptenboek, of iets heel anders. Dat domein gebruik je de rest van het blok voor het middagproject, dus kies iets wat genoeg variatie biedt om meerdere resources en relaties te ontdekken en wat je niet beu bent na middag 2.

Het startproject gebruikt Jakarta REST en draait op een embedded server. Persistence doe je met een H2 inmemory database, net als in eerdere modules. Testen kun je kiezen voor Postman of Bruno (naast unit tests).

Middag 1: Domein, resources en de eerste endpoints



Doel

We gaan beginnen met nadenken over je domein en je resources, voordat je ook maar één regel code schrijft. Je maakt keuzes die de rest van het project beïnvloeden: welke dingen bestaan er in jouw systeem, hoe noem je ze, en hoe geef je ze een plek in een URL?

Aan het eind van de middag heb je een draaiende API met je eerste echte endpoints, en heb je die al live getest vanuit Postman of Bruno.

Leerdoelen

- Een domeinmodel definiëren en vertalen naar REST-resources met betekenisvolle URL's
- Werkende GET-endpoints implementeren met Jakarta REST (`@GET`, `@Path`, `@Produces`)
- Een HTTP-response inspecteren in Postman: status, headers en body analyseren
- Beredeneren waarom een resource-georiënteerde aanpak verschilt van een actie-georiënteerde aanpak

Projectonderdelen

De middag bevat **3 onderdelen**.

Onderdeel 1: Domein bepalen en eerste endpoint live zetten

Begin met het kiezen van een domein als groep. Schrijf op welke resources je hebt: dat zijn de "dingen" in jouw systeem (zelfstandige naamwoorden, geen werkwoorden). Denk na over welke twee of drie resources je de komende weken gaat bouwen, en welke relaties daartussen bestaan.

Kies daarna één resource om vandaag mee te beginnen. Maak een Java record voor die resource en voeg de eerste twee endpoints toe:

- `GET /<jouw-resource>` (geeft alle items terug)
- `GET /<jouw-resource>/{id}` (geeft één item terug obv ID)

Gebruik het api-starter project als vertrekpunt. Verwijder de voorbeeldcode als je je eigen resource hebt.

Succescriteria

- Het domeinmodel is opgeschreven: minimaal twee resources met een beschrijving
- `GET /<resource>` geeft een JSON-array terug met minimaal drie items
- `GET /<resource>/{id}` geeft het juiste item terug bij een bestaand ID
- Beide endpoints zijn werkend getest in Postman of Bruno
- De response bevat een `Content-Type: application/json` header

Onderdeel 2: Tweede resource toevoegen

Voeg een tweede resource toe aan je API. Deze resource hoeft nog geen relatie te hebben met de eerste (dat komt later wel). Zorg wel dat de URL-structuur consistent is met de eerste resource.

Test ook wat er nu terugkomt als je een ID opvraagt dat niet bestaat. Wat is het gedrag? Zou je dit willen veranderen? (Hoef je nu nog niet zo te implementeren.)

Succescriteria:

- De tweede resource heeft ook een `GET all` en `GET by ID` endpoint
- Beide endpoints zijn getest en werken
- Je hebt genoteerd wat er terugkomt bij een niet-bestaand ID, en kunt uitleggen of dat gewenst gedrag is

Onderdeel 3: Postman-collectie aanmaken

Maak een Postman- of Bruno-collectie aan voor je API. Voeg alle requests die je vandaag hebt gebouwd toe als aparte requests in de collectie.

Voeg ook minstens één request toe dat bewust fout gaat: een ID dat niet bestaat, een verkeerd pad, of een methode die niet ondersteund wordt. Inspecteer de response aandachtig: welke statuscode krijg je? Wat staat er in de body?

Succescriteria

- Collectie bevat alle werkende requests van vandaag
- Minstens één "fout" request is toegevoegd en gedocumenteerd

Elk request heeft een duidelijke naam zodat anderen begrijpen wat het doet

Bonuschallenge (Optioneel)

Voeg een Postman of Bruno environment toe voor je lokale omgeving, zodat de base URL (<http://localhost:8080>) een variabele is in plaats van een hardcoded waarde in elk request. Bedenk waarom dit nuttig is zodra je de API ook ergens anders zou willen deployen.

Middag 2A: CRUD compleet met methods, statuscodes, foutafhandeling en unit tests (tot 15.15)

Let op er is een 2B!



Doel

Een API met alleen GET endpoints is handig voor het lezen van data, maar een echte API moet ook kunnen wijzigen. Vandaag bouw je de rest van de CRUD operaties: aanmaken, aanpassen en verwijderen. En we moeten natuurlijk foutafhandeling toevoegen. Want een ontvanger die een cryptische fout of een lege body met statuscode 500 terugkrijgt, weet niet wat hij moet doen.

Leerdoelen

- POST, PUT, PATCH en DELETE implementeren met de juiste semantiek en statuscodes
- Onderscheid maken tussen validatie-, business- en systeemfouten en ze anders afhandelen
- Een consistente foutstructuur ontwerpen die door de hele API hetzelfde is
- Unit tests schrijven voor de service-laag los van HTTP-afhandeling
- Schrijven van documentatie

Projectonderdelen

Dit middagonderdeel bevat **4 onderdelen**.

Onderdeel 1: Volledige CRUD implementeren

Voeg de resterende endpoints toe voor beide resources:

- `POST /<resource>` (nieuw item aanmaken)
- `PUT /<resource>/ {id}` (bestaand item volledig vervangen)
- `DELETE /<resource>/ {id}` (item verwijderen)

Gebruik de juiste statuscodes: **201 Created** bij aanmaken (met een **Location**-header die verwijst naar het nieuwe item), **200 OK** of **204 No Content** bij andere operaties, en **404 Not Found** als het item niet bestaat.

Succescriteria

- Alle CRUD endpoints zijn aanwezig voor beide resources
- **POST** geeft **201** terug met een **Location**-header
- **DELETE** op een niet-bestaand ID geeft **404** terug, niet **200**
- Alle endpoints zijn getest in de Postman-collectie

Onderdeel 2: Consistente foutafhandeling

Zorg dat foutmeldingen overal in je API dezelfde structuur hebben. Een goede foutresponse bevat minimaal:

```
{
  "status": 404,
  "error": "Not Found",
  "message": "Item met ID 42 bestaat niet"
}
```

Geen stacktraces in de response (die zijn voor de server-logs, niet voor de API-gebruiker, iets met security).

Behandel minstens drie foutgevallen expliciet:

- Resource niet gevonden (404)
- Ongeldige input van de gebruiker (400)
- Een generieke serverfout (500) waarbij de interne details verborgen blijven

Succescriteria

- Alle foutresponses volgen dezelfde JSON structuur
- Stacktraces verschijnen niet in de responsebody
- De drie foutgevallen zijn getest met Postman requests die de fout bewust triggeren
- De Postman-collectie bevat een sectie "unhappy paths"

Onderdeel 3: Unit tests voor de service-laag

Voeg een service klasse toe die de businesslogica bevat, los van de Jakarta REST resource klasse. Schrijf unit tests voor de service met JUnit.

Test minimaal:

- Ophalen van een item dat bestaat
- Ophalen van een item dat niet bestaat (verwachte exceptie of lege Optional)
- Aanmaken van een item met geldige invoer

Gebruik mocks voor eventuele afhankelijkheden (bijv. de data laag), zodat de tests snel zijn en niets schrijven naar de database.

Succescriteria

- Er is een aparte service klasse naast de resource klasse
- Minimaal drie unit tests voor de service
- Tests draaien zonder een draaiende server of database
- Tests slagen bij `mvn test`

Onderdeel 4: Documentatie voor ander team

Zorg dat je documentatie hebt voor het andere team zodat zij jullie API kunnen aanroepen met de juiste informatie en weten wat zij terug kunnen verwachten.

Middag 2B: API van ander team gebruiken (tot 16.15)

Doel

Documentatie schrijven is leuk, maar de echte test is of een ander team er iets mee kan. Dat is wat we nu gaan doen. Een ander team gaat met jullie documentatie aan de slag en jullie gaan met de documentatie van het andere team werken. Je bouwt een client die de API van een ander team aanroept en doet dit puur op basis van de documentatie, zonder dat je het tussendoor kunt testen of de code mag inzien. (Net als in het echt, waar je vaak integreert tegen een API die nog niet draait of waar je (nog) geen toegang toe hebt.)

Leerdoelen

- Een API consumeren die je niet zelf hebt gebouwd, op basis van een contract (de documentatie)
- Een Postman-collectie of Java-client opbouwen die method, pad, headers en body correct invult
- Ervaren waarom complete, ondubbelzinnige documentatie belangrijk is

Projectonderdelen

De middagdeel bevat **3 onderdelen**.

Onderdeel 1: Documentatie uitwisselen

Geef je eigen documentatie aan een ander team. Dit is de verdeling:

- Team Pokemon/kitty geeft documentatie aan Team Grolsch.
- Team Grolsch geeft documentatie aan Team home.
- Team home geeft documentatie aan Team Pokemon/kitty.

Niet toegestaan om source code uit te delen of overige vragen te beantwoorden.

Onderdeel 2: Maak een client op basis van de documentatie

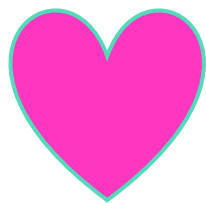
Maak op basis van de ontvangen documentatie een Postman-collectie (of Java-code) die hun API aanroept. Neem alle endpoints op: de happy paths (GET, POST, PUT, DELETE) en minstens één unhappy path (bijv. bewust een 404 of 400 triggeren). Geef aan wat je terug verwacht.

Je kunt dit nu niet live testen want hun API draait nog niet live. Werk dus strikt volgens de documentatie. Loop je tegen iets onduidelijks aan? Noteer het; dat is waardevolle feedback voor het andere team

Onderdeel 3: De test (aka moment suprême)

Dit gaan we live doen in de laatste 15 minuten. De verwachting is chaos, alles daarboven is fantastisch. Stuur je Postman-collectie of Java code naar het team wiens API je hebt aangeroepen. Zij draaien hun API en voeren jouw requests uit. Andersom ontvang jij de client van het team aan wie jij je documentatie gaf, en draai je jouw eigen API ertegen terwijl je het scherm deelt. Werkt het in één keer? Dan was de documentatie goed (en het team kundig natuurlijk). Werkt het niet? Bespreek kort waar het fout gegaan is (indien snel duidelijk, want tijd).

Enjoy en fijne zomer



Middag 3: API design principes, review en integration tests



Doel

Vandaag gaan we even opnieuw even kijken naar wat we gemaakt hebben: kloppen de URLs? Zijn de naamgevingsconventies consistent? Zou een externe developer begrijpen wat jouw API doet?

Je voert een designreview uit op je eigen API, past aan waar nodig, en je schrijft integratietests die de HTTP-laag ook meenemen (iets wat unit tests op de service niet kunnen).

Leerdoelen

- Gemaakte designkeuzes evalueren aan de hand van REST designprincipes
- Een API refactoren op basis van naamgeving, consistentie en resource-modellering
- Integration tests schrijven die via HTTP testen (met REST Assured) en verklaren wat dat toevoegt boven unit tests
- Versiebeheer en backward compatibility beargumenteren

Projectonderdelen

De middag bevat **3 onderdelen**.

Onderdeel 1: Design review en refactoring

Loop je bestaande API door met de volgende vragen:

- Staan er werkwoorden in je URL-paden? (bijv. `/getItems` of `/createItem`)
- Zijn collection resources in het meervoud? (`/books`, niet `/book`)
- Zijn datumformats consistent? (gebruik ISO 8601: `2026-06-14`)
- Zijn response-keys in camelCase of snake_case (en is dat overal hetzelfde)?
- Wat geeft je API terug als een optioneel veld geen waarde heeft: `null` of ontbreekt het veld?

Pas aan wat er niet klopt. Documenteer welke wijzigingen je hebt gemaakt en waarom. Eventueel kun je versioning toepassen als bonus.

Succescriteria

- URLs bevatten geen werkwoorden
- Naamgeving is consistent door de hele API
- Alle wijzigingen zijn gedocumenteerd met een korte uitleg
- De Postman collectie is bijgewerkt na de refactoring

Onderdeel 2: Pagination en filtering

Voeg pagination toe aan je `GET all` endpoints. Gebruik query parameters:

```
GET /books?page=0&size=10
```

Geef in de response niet alleen de items terug, maar ook metadata:

```
{
  "content": [...],
  "page": 0,
  "size": 10,
  "totalElements": 42
}
```

Voeg optioneel ook een filter toe, bijvoorbeeld op naam of categorie:

```
GET /books?genre=fantasy
```

Succescriteria

- `GET all` ondersteunt `page` en `size` query parameters
- Response bevat paginatiemetadata
- Een test in Postman vraagt pagina 0 met size 2 op en krijgt maximaal 2 items terug
- Bij ontbrekende parameters wordt een zinvolle default gebruikt

Onderdeel 3: Integration tests met REST Assured

Schrijf integration tests die via HTTP testen tegen een draaiende in-memory instantie van je API. Gebruik REST Assured.

Voeg de dependency toe aan je `pom.xml`:

```
<dependency>

    <groupId>io.rest-assured</groupId>

    <artifactId>rest-assured</artifactId>

    <scope>test</scope>

</dependency>
```

Schrijf tests voor minimaal:

- Happy path: haal een bestaand item op en controleer statuscode en body
- Unhappy path: vraag een niet-bestaand ID op en controleer dat je 404 krijgt
- POST: maak een nieuw item aan en controleer dat het teruggevonden kan worden

Succescriteria

- Integration tests draaien met `mvn test`
- Tests controleren statuscode, `Content-Type` header en JSON-body
- Er is minstens één test per scenario (happy / unhappy / POST)
- Je kunt uitleggen wat deze tests toevoegen boven de service-unit-tests van gisteren

Bonuschallenge (Optioneel)

Voeg API versioning toe via de URL: `/api/v1/books`. Bedenk wat backward compatibility betekent voor jouw API: welke wijzigingen kun je maken zonder bestaande clients te breken? Welke wijzigingen vereisen een nieuwe versie?

Middag 4: Authenticatie en basic auth



Doel

Vandaag voeg je authenticatie toe: de server weet wie de aanroeper is. En met die kennis kun je ook autorisatie gaan doen: de server bepaalt wat de aanroeper mag.

We beginnen met Basic Auth. Dit is simpel, goed te begrijpen en handig om de concepten mee te oefenen. We schrijven daarna tests die bewijzen dat de beveiliging werkt. Dat betekent dat de tests moeten aantonen dat een niet-geauthenticeerde aanroeper geen toegang krijgt, en dat een gebruiker niet bij de data van een ander kan.

Leerdoelen

- Basic Auth implementeren als authenticatiemechanisme voor een REST API
- Het verschil tussen authenticatie en autorisatie toepassen in API-context
- Testen schrijven die 401 en 403 responses afdwingen
- Een IDOR-kwetsbaarheid herkennen en testen

Projectonderdelen

De middag bevat **3 onderdelen**.

Onderdeel 1: Basic Auth toevoegen

Beveilig een deel van je endpoints met Basic Auth. Endpoints die data wijzigen (POST, PUT, DELETE) mogen alleen toegankelijk zijn voor geauthenticeerde gebruikers. GET-endpoints mogen voor iedereen toegankelijk blijven.

Sla o.a. wachtwoorden niet op als plaintext. Gebruik een gehashte opslag (bcrypt of minimaal SHA-256 via [MessageDigest](#)).

Implementeer een `ContainerRequestFilter` die de `Authorization`-header uitleest en valideert. Geef een `401 Unauthorized` terug bij een ontbrekende of ongeldige header, met een `WWW-Authenticate: Basic realm="api"` header in de response.

Succescriteria

- POST, PUT en DELETE vereisen een geldige `Authorization: Basic ...` header
- `GET` endpoints zijn bereikbaar zonder authenticatie
- Een request zonder header geeft `401` terug, met de juiste `WWW-Authenticate` header
- Wachtwoorden staan niet als plaintext opgeslagen in de code of configuratie
- Getest in Postman oid: zowel met als zonder credentials

Onderdeel 2: Role based autorisatie

Voeg twee rollen toe: een gewone gebruiker en een admin. Een admin mag alles. Een gewone gebruiker mag alleen de eigen data aanpassen.

Implementeer minstens één endpoint waarbij de rol bepaalt of de aanroep is toegestaan. Bijv. een admin kan elke resource verwijderen, een gewone gebruiker mag dat niet. Geef `403 Forbidden` terug bij onvoldoende rechten.

Succescriteria

- Er zijn minimaal twee rollen gedefinieerd
- Een admin gebruiker en een gewone gebruiker zijn aanwezig in de testdata
- `DELETE` met een admin account geeft `200/204` terug
- `DELETE` met een gewone gebruiker geeft `403` terug
- Getest in Postman met beide gebruikers

Onderdeel 3: Security tests

Schrijf automatische security tests die bewijzen dat je beveiliging werkt. Gebruik REST Assured.

Schrijf tests voor:

- Aanroep zonder credentials geeft `401`
- Aanroep met verkeerde credentials geeft `401`
- Gewone gebruiker die een admin-actie probeert geeft `403`

- IDOR-test: gebruiker A kan de data van gebruiker B niet aanpassen (als jouw domein dat heeft)

Succescriteria

- Alle vier testscenario's zijn geïmplementeerd
- Tests draaien met `mvn test`
- Je kunt uitleggen wat IDOR is en of jouw API er gevoelig voor zou zijn

Bonuschallenge (Optioneel)

Genereer een OpenAPI/Swagger-specificatie voor je API. Je kunt [smallrye-open-api](#) of een vergelijkbare library gebruiken met Jakarta REST. Controleer of de beveiligde endpoints correct als "secured" worden weergegeven in de gegenereerde documentatie. Is de specificatie volledig en klopt het met hoe jouw API werkelijk gedraagt?

Middag 5: JWT en Geavanceerde Autorisatie



Doel

Basic Auth werkt, maar het heeft beperkingen. Je stuurt bijvoorbeeld bij elke request de gebruikersnaam en het wachtwoord mee. Bij JWT werkt dat anders: je authenticceert één keer en krijgt een token terug, waarmee je vervolgens requests doet. De server hoeft het wachtwoord dan niet bij elke request te controleren.

Vandaag vervang je Basic Auth door JWT-authenticatie. Jullie hoeven het token-uitgiftesysteem niet zelf te bouwen. Je kunt het systeem van de ochtend gebruiken. Jullie taak is om dat token te valideren in je API en er autorisatiebeslissingen op te baseren.

Leerdoelen

- JWT-tokens valideren in een Jakarta REST filter
- Claims uit een JWT uitlezen en gebruiken voor autorisatie
- Rolgebaseerde en resource gebaseerde autorisatie implementeren
- Security tests schrijven die verschillende rollen en tokens testen

Projectonderdelen

De middag bevat **3 onderdelen**.

Onderdeel 1: JWT authenticatie

Vervang de Basic Auth-filter door een JWT-filter.

De filter leest de `Authorization: Bearer <token>` header uit, valideert de handtekening van het token met de geleverde public key, en slaat de claims op zodat de rest van de applicatie er gebruik van kan maken.

Geef `401 Unauthorized` terug als het token ontbreekt, verlopen is of ongeldig is.

Succescriteria

- De API accepteert een geldig JWT-token in de `Authorization: Bearer` header
- Een verlopen of ongeldig token geeft `401`
- Een ontbrekende header geeft `401`
- De filter is getest met zowel een geldig als een ongeldig token

Onderdeel 2: Rolgebaseerde autorisatie op basis van JWT-claims

Gebruik de claims in het token (bijv. een `roles` claim) om autorisatie te bepalen. Pas de bestaande autorisatielogica aan zodat rollen uit het token worden gelezen in plaats van uit een hardcoded gebruikerslijst.

Zorg dat:

- Endpoints die alleen voor admins zijn, een `403` geven voor tokens zonder de admin rol
- Endpoints die eigen data beschermen, controleren of de `sub`-claim (of een andere gebruikersID in het token) overeenkomt met het gevraagde resource eigenaarschap

Succescriteria

- Rollen worden uitgelezen uit de JWT claims
- Admin endpoints geven `403` bij een token zonder admin rol
- Gebruiker A kan de data van gebruiker B niet aanpassen (IDOR-bescherming)
- De hardcoded gebruikerslijst is verwijderd of niet meer leidend

Onderdeel 3: Security tests voor meerdere rollen

Breid de security tests uit voor het nieuwe JWT systeem. Gebruik de aangeleverde tool om tokens te genereren voor verschillende rollen.

Schrijf tests voor:

- Admintoken kan admin acties uitvoeren
- Gebruikertoken krijgt `403` op admin acties
- Verlopen token geeft `401`
- Token van gebruiker A kan niet de data van gebruiker B wijzigen

Succescriteria

- Alle vier testscenario's zijn geïmplementeerd met REST Assured
- Tests gebruiken tokens die gegenereerd zijn met het aangeleverde systeem
- Tests draaien met `mvn test` en slagen

